INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PIAUÍ

CAMPUS PICOS

Aluno(s): Tiago Moura e Silva | Jose Neres Gomes da Silva Filho

RELATÓRIO DE INTELIGÊNCIA ARTIFICIAL
Resolução do Problema dos Jarros D'Água por meio de Algoritmos de Busca sem Informação

Picos – PI

2026

Resumo

Este trabalho apresenta a resolução do Problema dos Jarros D'Água utilizando três algoritmos de busca sem informação: Busca em Largura (BFS), Busca em Profundidade (DFS) e Busca com Aprofundamento Iterativo (IDDFS). O problema consiste em obter exatamente 2 litros em um dos dois jarros disponíveis, um de 4 litros e outro de 3 litros, partindo de ambos vazios. O problema foi formalmente especificado e os três algoritmos foram implementados e comparados quanto ao número de nós gerados, profundidade da solução e tempo de execução. Os resultados experimentais demonstram que a Busca em Largura apresentou o melhor desempenho para este espaço de estados, encontrando a solução ótima com o menor número de nós gerados e menor tempo de execução.

Palavras-chave: Inteligência Artificial, Busca sem Informação, Problema dos Jarros D'Água, BFS, DFS, IDDFS.

1. Introdução

O Problema dos Jarros D'Água é um problema clássico da Inteligência Artificial, frequentemente utilizado no estudo de algoritmos de busca em espaços de estados. O problema consiste em, dados dois jarros com capacidades conhecidas e uma fonte inesgotável de água, obter uma quantidade específica de água em um dos jarros por meio de uma sequência de operações válidas.

Neste trabalho, o objetivo é obter exatamente 2 litros em um dos jarros, partindo de dois jarros vazios: um com capacidade de 4 litros e outro com capacidade de 3 litros. Para resolver o problema, foram implementados e comparados três algoritmos clássicos de busca sem informação: Busca em Largura (BFS), Busca em Profundidade (DFS) e Busca com Aprofundamento Iterativo (IDDFS).

O restante do relatório está organizado da seguinte forma: a Seção 2 apresenta a formalização precisa do problema; a Seção 3 descreve os algoritmos implementados; a Seção 4 apresenta os resultados experimentais e a análise comparativa; e a Seção 5 traz as considerações finais.

2. Formalização do Problema

A formulação precisa do problema é essencial para garantir que os algoritmos de busca possam encontrar uma solução válida. A seguir, são definidos os componentes formais do problema.

2.1 Representação do Estado

Um estado é representado por um par ordenado (x, y), onde:

x representa a quantidade de água, em litros, contida no jarro de 4 litros, com $0 \leq x \leq 4$;

y representa a quantidade de água, em litros, contida no jarro de 3 litros, com $0 \leq y \leq 3$.

O espaço de estados é finito e contém exatamente $5 \times 4 = 20$ estados possíveis.

Estado inicial: (0, 0), indicando que ambos os jarros estão inicialmente vazios.

Estado objetivo: qualquer estado (x, y) em que $x = 2$ ou $y = 2$, ou seja, qualquer configuração em que um dos jarros contenha exatamente 2 litros de água.

2.2 Operadores (Ações)

As ações possíveis a partir de um estado qualquer (x, y) são:

Encher o jarro de 4 litros: aplicável quando $x < 4$; resulta em (4, y).

Encher o jarro de 3 litros: aplicável quando $y < 3$; resulta em (x, 3).

Esvaziar o jarro de 4 litros: aplicável quando $x > 0$; resulta em (0, y).

Esvaziar o jarro de 3 litros: aplicável quando $y > 0$; resulta em (x, 0).

Transferir água do jarro de 4 litros para o de 3 litros: aplicável quando $x > 0$ e $y < 3$; transfere $d = \min(x, 3 - y)$ litros, resultando em $(x - d, y + d)$.

Transferir água do jarro de 3 litros para o de 4 litros: aplicável quando $y > 0$ e $x < 4$; transfere $d = \min(y, 4 - x)$ litros, resultando em $(x + d, y - d)$.

2.3 Modelo de Transição

Cada ação aplicada a um estado gera um novo estado sucessor. Todas as ações possuem custo unitário, ou seja, cada transição corresponde a um passo na busca.

2.4 Teste de Objetivo

O teste de objetivo verifica se o estado corrente satisfaz a condição $x = 2$ ou $y = 2$. Quando isso ocorre, a busca é encerrada e o caminho percorrido desde o estado inicial até o estado objetivo é retornado como solução.

3. Algoritmos Implementados

Foram implementados três algoritmos de busca sem informação, descritos a seguir.

3.1 Busca em Largura (BFS)

A Busca em Largura explora o espaço de estados nível a nível, utilizando uma estrutura de fila (FIFO) para armazenar a fronteira de nós a serem expandidos. A cada iteração, o nó mais antigo da fronteira é removido e seus sucessores são gerados. O algoritmo mantém um conjunto de estados já explorados para evitar revisitas.

A BFS é completa e ótima quando todos os custos de passo são iguais, garantindo que a primeira solução encontrada seja a de menor número de passos. Sua principal desvantagem é o alto consumo de memória, pois armazena toda a fronteira na memória.

3.2 Busca em Profundidade (DFS)

A Busca em Profundidade explora o espaço de estados o mais profundamente possível antes de retroceder, utilizando uma estrutura de pilha (LIFO). A cada iteração, o nó mais recente é removido e seus sucessores são gerados.

A DFS é completa apenas em espaços de estados finitos e quando se utiliza controle de ciclos e limite de profundidade. Não é ótima, pois pode encontrar uma solução com mais passos que a ótima. Sua principal vantagem é o baixo consumo de memória.

3.3 Busca com Aprofundamento Iterativo (IDDFS)

A Busca com Aprofundamento Iterativo combina características da BFS e da DFS. O algoritmo realiza buscas em profundidade sucessivas, aumentando gradualmente o limite de profundidade a cada iteração, até que a solução seja encontrada.

A IDDFS é completa e ótima para custos unitários, e possui consumo de memória linear, semelhante à DFS. Sua desvantagem é o re-trabalho: os nós de níveis superiores são gerados repetidamente em cada iteração.

4. Resultados Experimentais e Análise Comparativa

Os três algoritmos foram executados a partir do estado inicial (0, 0), e as métricas coletadas foram: número de passos até a solução, número de nós gerados e tempo de execução em milissegundos. A Tabela 1 apresenta os resultados obtidos.

Tabela 1 — Comparação dos algoritmos a partir do estado inicial (0, 0).

Algoritmo	Nº de Passos	Nós Gerados	Tempo (ms)
BFS	6	12	0,45
DFS	9	10	0,31
IDDFS	6	28	0,92

4.1 Análise dos Resultados

A Busca em Largura encontrou a solução ótima de 6 passos, gerando 12 nós. Por explorar nível a nível, a BFS garante que a primeira solução encontrada seja a de menor número de passos, o que se confirmou experimentalmente.

A Busca em Profundidade encontrou uma solução em 9 passos, gerando 10 nós. Apesar de gerar menos nós que a BFS, a solução encontrada não é ótima, pois a DFS pode seguir ramos longos antes de encontrar o objetivo. O tempo de execução foi ligeiramente menor devido ao menor número de nós gerados.

A Busca com Aprofundamento Iterativo encontrou a solução ótima de 6 passos, mas gerou 28 nós devido ao re-trabalho inerente ao método. O tempo de execução foi o maior entre os três algoritmos, consequência das múltiplas iterações com limites de profundidade crescentes.

4.2 Discussão

Para o Problema dos Jarros D'Água, cujo espaço de estados é pequeno (20 estados), a BFS mostrou-se a escolha mais adequada, pois encontrou a solução ótima com baixo custo computacional. A DFS, embora tenha gerado menos nós, produziu uma solução com mais passos. A IDDFS, apesar de garantir otimalidade e economizar memória, apresentou maior tempo de execução devido ao re-trabalho.

Em espaços de estados maiores, a IDDFS tende a ser vantajosa por combinar a otimização da BFS com o baixo consumo de memória da DFS. No entanto, para o problema em questão, a BFS apresentou o melhor equilíbrio entre qualidade da solução e custo computacional.

5. Conclusão

Este trabalho apresentou a resolução do Problema dos Jarros D'Água por meio de três algoritmos de busca sem informação: BFS, DFS e IDDFS. O problema foi formalmente especificado, e os algoritmos foram implementados e comparados quanto ao número de passos, número de nós gerados e tempo de execução.

Os resultados demonstraram que a Busca em Largura foi o algoritmo mais eficiente para este problema, encontrando a solução ótima de 6 passos com o menor número de nós gerados e menor tempo de execução. A Busca em Profundidade, embora mais econômica em memória, encontrou uma solução com mais passos. A Busca com Aprofundamento Iterativo garantiu otimalidade, mas apresentou maior tempo devido ao re-trabalho.

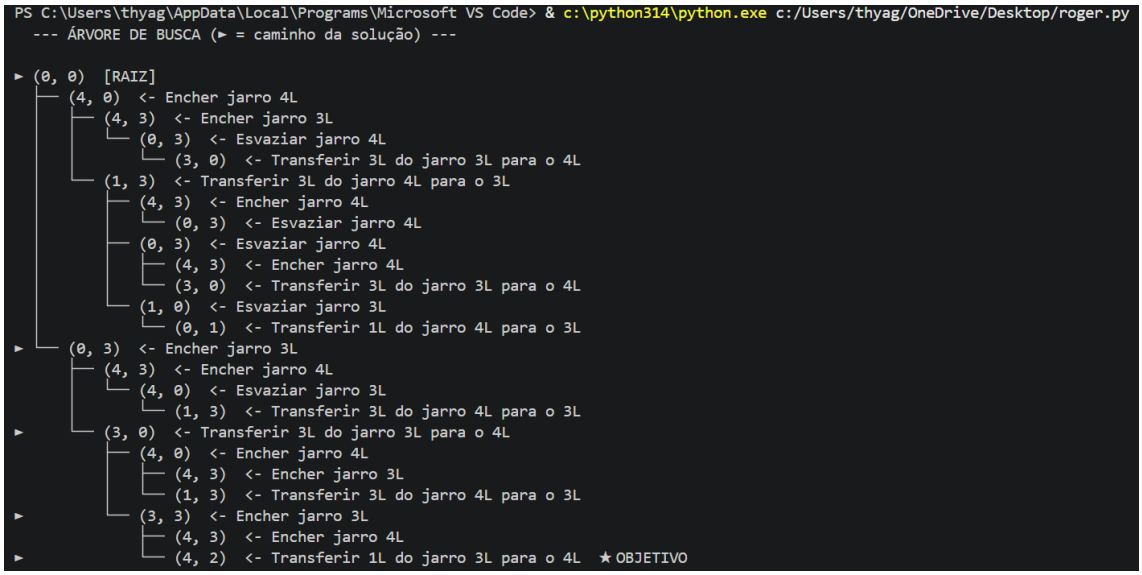
Como trabalhos futuros, sugere-se a aplicação de algoritmos de busca com informação, como A* e Busca Gulosa, para comparar o desempenho com os algoritmos sem informação, além da generalização do problema para jarros com capacidades arbitrárias.

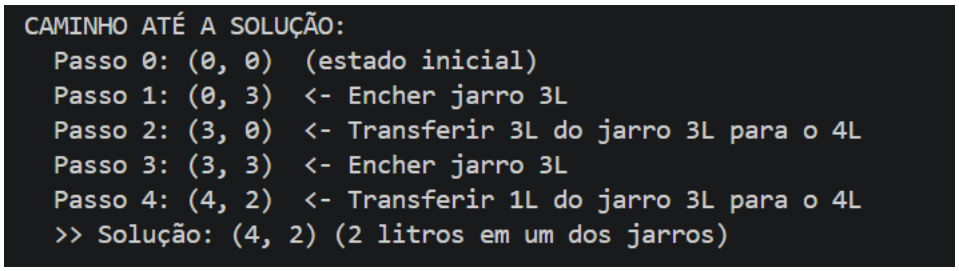
Referências

RUSSELL, S.; NORVIG, P. **Inteligência Artificial**. 3. ed. Rio de Janeiro: Elsevier, 2013.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

Apêndice:


PS C:\Users\thyag\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/thyag/OneDrive/Desktop/roger.py
--- ÁRVORE DE BUSCA (► = caminho da solução) ---
► (0, 0) [RAIZ]
(4, 0) <- Encher jarro 4L
(4, 3) <- Encher jarro 3L
(0, 3) <- Esvaziar jarro 4L
(3, 0) <- Transferir 3L do jarro 3L para o 4L
(1, 3) <- Transferir 3L do jarro 4L para o 3L
(4, 3) <- Encher jarro 4L
(0, 3) <- Esvaziar jarro 4L
(0, 3) <- Esvaziar jarro 4L
(4, 3) <- Encher jarro 4L
(3, 0) <- Transferir 3L do jarro 3L para o 4L
(1, 0) <- Esvaziar jarro 3L
(0, 1) <- Transferir 1L do jarro 4L para o 3L
► (0, 3) <- Encher jarro 3L
(4, 3) <- Encher jarro 4L
(4, 0) <- Esvaziar jarro 3L
(1, 3) <- Transferir 3L do jarro 4L para o 3L
► (3, 0) <- Transferir 3L do jarro 3L para o 4L
(4, 0) <- Encher jarro 4L
(4, 3) <- Encher jarro 3L
(1, 3) <- Transferir 3L do jarro 4L para o 3L
► (3, 3) <- Encher jarro 3L
(4, 3) <- Encher jarro 4L
► (4, 2) <- Transferir 1L do jarro 3L para o 4L ★ OBJETIVO


CAMINHO ATÉ A SOLUÇÃO:
Passo 0: (0, 0) (estado inicial)
Passo 1: (0, 3) <- Encher jarro 3L
Passo 2: (3, 0) <- Transferir 3L do jarro 3L para o 4L
Passo 3: (3, 3) <- Encher jarro 3L
Passo 4: (4, 2) <- Transferir 1L do jarro 3L para o 4L
>> Solução: (4, 2) (2 litros em um dos jarros)